ARKA-Lite

Proyecto final · Upskilling Backend Java

Una UI y dos microservicios Spring Boot: lo que aprendimos y cómo funciona.

RECORRIDO EJECUTIVO · CONTENIDO Y GUION

PARTES

1. **Fundamentos** Spring Boot, REST, reglas de negocio, pruebas y Git.
2. **Arquitectura** Microservicios, puertos y adaptadores, y un contrato por eventos.
3. **Base de datos** Una base por servicio, un esquema que se cuida solo y datos consistentes.
4. **Contenedores y Kubernetes** El mismo sistema en cualquier computadora y un clúster que se repara solo.
5. **CI/CD** Integración y entrega continua: cada cambio se prueba y se empaqueta solo.
6. **Observabilidad** Logs centralizados, trazas entre servicios y salud de cada componente.
7. **Documentación** Contratos que salen del código y documentación que cambia con él.
8. **La UI como laboratorio** No es parte del entregable: es el banco de pruebas para ver cómo se comporta el sistema, sobre todo cuando algo falla.

Generado desde docs/presentation-data/contenido. Cada lámina tiene: lo que se ve, el diagrama detallado cuando lo hay, y el guion: qué decir, qué hacer, si preguntan y para pasar.

Apertura

1. Del endpoint al clúster

EN PANTALLA

- Lo que aprendimos construyendo dos microservicios Java, de punta a punta.

QUÉ DECIR

“Buenos días. Hoy les presentamos ARKA-Lite, el proyecto final del curso Upskilling Backend Java.”

“Empezamos escribiendo un endpoint y terminamos con un sistema de dos microservicios corriendo en Kubernetes, con su pipeline, su base de datos y su observabilidad. No les vamos a contar el temario: les vamos a contar qué construimos, qué medimos y qué aprendimos.”

QUÉ HACER (NO SE DICE)

- Antes de empezar, abrir la UI en http://localhost:8088 y comprobar que los dos servicios aparecen en verde.

PARA PASAR

“Primero, el proyecto en números.”

2. ARKA-Lite en números

EN PANTALLA · NÚMEROS GRANDES

- 2 microservicios: Spring Boot 4 y Java 26, cada uno con su PostgreSQL.
- 11 endpoints REST: Documentados con OpenAPI y Swagger.
- 67 pruebas automáticas: 28 en solicitudes, 10 en notificaciones, 29 en la UI.
- 8 contenedores: Servicios, bases, UI y logs con un solo comando.
- <= 5 s para entregar un aviso: El relevador revisa los pendientes cada 5 segundos.
- 0 avisos perdidos o duplicados: Aunque el otro servicio esté caído o el evento llegue dos veces.

IDEAS CLAVE

- Dos microservicios con su propia base
- Un solo comando levanta todo
- Cada cambio pasa por pruebas automáticas

QUÉ DECIR

“ARKA-Lite hace algo sencillo a propósito: registra solicitudes, como una incidencia o un pedido de cambio, y avisa cuando una solicitud se envía. El problema es pequeño para que el foco esté en cómo se construye, se despliega y se opera.”

“Son dos microservicios con Spring Boot, once endpoints documentados, sesenta y siete pruebas automáticas y ocho contenedores que se levantan con un solo comando.”

“Y los dos números que más nos importan: un aviso se entrega en cinco segundos o menos, y no se pierde ni se duplica ninguno, aunque el otro servicio esté caído. Más adelante les mostramos cómo lo logramos.”

PARA PASAR

“Antes de entrar, ocho palabras que vamos a usar mucho.”

---

3. Ocho conceptos, en una frase

EN PANTALLA · TARJETAS

- Microservicio: Una aplicación pequeña, con una sola tarea y su propia base de datos.
- API REST: La puerta para pedirle algo a una aplicación por HTTP, como abrir una URL.
- Arquitectura hexagonal: El negocio al centro; la base y la web se conectan por “enchufes”.
- ORM (JPA): Traduce entre clases Java y tablas: escribe el SQL rutinario por nosotros.
- Docker: Empaqueta la aplicación con todo lo que necesita: corre igual en cualquier lado.
- Kubernetes: Administra esos paquetes: los arranca, los vigila y los reinicia si fallan.
- CI/CD: Un robot que compila, prueba y empaqueta cada cambio, sin pasos manuales.
- Observabilidad: Logs y salud de todo el sistema en un solo lugar.

IDEAS CLAVE

- Microservicio y API REST
- Arquitectura hexagonal y ORM
- Docker y Kubernetes
- CI/CD y observabilidad

QUÉ DECIR

“Sabemos que muchos de ustedes trabajan sobre todo con la base de datos y procedimientos almacenados, así que antes de entrar les dejamos ocho conceptos en una frase.”

“Un microservicio es una aplicación pequeña con una sola tarea y su propia base. Una API REST es la puerta para pedirle cosas por HTTP. La arquitectura hexagonal pone el negocio al centro y conecta la base y la web como si fueran enchufes. Y un ORM traduce entre clases Java y tablas, para no escribir a mano el SQL de todos los días.”

“Docker empaqueta la aplicación con todo lo que necesita; Kubernetes administra esos paquetes. CI/CD es un robot que prueba cada cambio. Y observabilidad es poder ver qué pasa adentro. Cada uno lo vamos a ver con un ejemplo.”

SI PREGUNTAN

¿Esto reemplaza a los procedimientos almacenados?

No necesariamente. Mueve las reglas del negocio al código; el SQL pesado o los reportes pueden seguir en la base.

PARA PASAR

“Este es el recorrido.”

---

4. Qué vamos a ver

EN PANTALLA · LA RUTA DE LAS PARTES

- 1. Fundamentos: Spring Boot, REST, reglas de negocio, pruebas y Git.
- 2. Arquitectura: Microservicios, puertos y adaptadores, y un contrato por eventos.
- 3. Base de datos: Una base por servicio, un esquema que se cuida solo y datos consistentes.
- 4. Contenedores y Kubernetes: El mismo sistema en cualquier computadora y un clúster que se repara solo.
- 5. CI/CD: Integración y entrega continua: cada cambio se prueba y se empaqueta solo.
- 6. Observabilidad: Logs centralizados, trazas entre servicios y salud de cada componente.
- 7. Documentación: Contratos que salen del código y documentación que cambia con él.
- 8. La UI como laboratorio: No es parte del entregable: es el banco de pruebas para ver cómo se comporta el sistema, sobre todo cuando algo falla.

IDEAS CLAVE

- Fundamentos y arquitectura
- Base de datos
- Contenedores, CI/CD y observabilidad
- Documentación, la UI y el valor

QUÉ DECIR

“Vamos en ocho partes, en el orden en que se construye un sistema: fundamentos, arquitectura, base de datos, contenedores y Kubernetes, integración y entrega continua, observabilidad y documentación.”

“Después les mostramos la interfaz que usamos como laboratorio, sobre todo para ver qué pasa cuando algo falla. Y cerramos con el valor que esto deja.”

“Cada parte termina con una lámina de aprendizajes: qué aprendimos, cómo lo aplicamos y qué valor tiene para un equipo de tecnología.”

PARA PASAR

“Empecemos por los fundamentos.”

1 · Fundamentos

Spring Boot, REST, reglas de negocio, pruebas y Git.

5. Cómo funciona Spring Boot

EN PANTALLA · FLUJO PASO A PASO

- 1. main(): SpringApplication.run arranca la aplicación.
- 2. Autoconfiguración: Ve web, JPA y PostgreSQL y los configura solos.
- 3. Beans e inyección: Crea cada pieza y le pasa lo que necesita.
- 4. Servidor embebido: Tomcat queda escuchando en el puerto 8080.
- 5. Controlador: Recibe el JSON, llama al negocio y responde.

IDEAS CLAVE

- Un main arranca todo
- Se configura solo según las librerías
- Crea las piezas y las conecta por el constructor
- Trae su propio servidor web

Comparación: Como un contratista de obra: le decimos qué piezas hay y él las arma, las conecta y abre la puerta.

QUÉ DECIR

"Antes de Spring Boot, levantar una aplicación Java web significaba instalar un servidor, configurar la conexión a la base y conectar cada clase a mano. Spring Boot hace ese trabajo."

"Todo empieza en un main. Spring Boot mira qué librerías tenemos, por ejemplo web, JPA y el driver de PostgreSQL, y las configura solo. Luego crea cada pieza de la aplicación, lo que en Spring se llama un bean, y le pasa por el constructor lo que necesita: eso es la inyección de dependencias."

"Al final arranca su propio servidor web, Tomcat, en el puerto 8080. Cuando llega una petición, el controlador la recibe, llama al negocio y responde. Nosotros solo escribimos el negocio y decimos qué piezas hay."

QUÉ HACER (NO SE DICE)

- Mostrar: Ejecutar ./mvnw clean verify dentro de servicio-solicitudes. Debe verse: BUILD SUCCESS y el resumen de pruebas.

SI PREGUNTAN

¿Qué es un bean?

Un objeto que Spring crea y administra, y que puede pasarle a otros por el constructor.

PARA PASAR

"Con la aplicación arriba, veamos cómo responde la API."

6. Una API que responde siempre igual

EN PANTALLA · NÚMEROS GRANDES

- 201 Creada: La solicitud quedó en BORRADOR.
- 400 Faltan datos: El identificador o el tipo vienen vacíos.
- 404 No existe: No hay una solicitud con ese identificador.
- 409 Conflicto: El id ya existe o el cambio de estado no se permite.

IDEAS CLAVE

- Método + ruta = una operación
- La regla está en el dominio, sin Spring
- Un manejador global convierte errores en respuestas
- Siempre la misma forma: estado y mensaje

QUÉ DECIR

"Un endpoint REST es método más ruta: POST a /solicitudes crea, GET lista. El controlador solo traduce; quien decide es el dominio, que son clases Java normales, sin Spring. Por ejemplo, una solicitud solo se puede enviar si está en borrador."

"Si la regla no se cumple, el dominio lanza una excepción, y un único manejador global la convierte en una respuesta HTTP. Así cada error tiene su código: 400 si faltan datos, 404 si no existe, 409 si hay un conflicto. Siempre con la misma forma: estado y mensaje."

"Para el staff esto significa una API predecible: quien la consume sabe qué pasó sin leer el código."

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, crear dos veces el mismo identificador. Debe verse: El diálogo "Ese identificador ya existe" con el código 409.

PARA PASAR

"Que funcione no basta: el código también tiene que ser fácil de cambiar."

7. Código mantenible: buenas prácticas

EN PANTALLA · TABLA DE TRES COLUMNAS

- Práctica: Una responsabilidad por clase · Cómo la aplicamos: El controlador traduce, el servicio decide, el adaptador guarda · Por qué ayuda a mantener: Un cambio toca un solo lugar
- Práctica: Inyección por constructor · Cómo la aplicamos: Cada pieza recibe sus dependencias; nadie hace new de lo que necesita · Por qué ayuda a mantener: Cualquier pieza se reemplaza o se prueba sola
- Práctica: Datos inmutables · Cómo la aplicamos: La Solicitud es un record: cambiar de estado crea una nueva · Por qué ayuda a mantener: Sin estados a medias ni efectos ocultos
- Práctica: Errores en un solo lugar · Cómo la aplicamos: Excepciones del dominio y un manejador global · Por qué ayuda a mantener: Respuestas iguales, sin try/catch repetidos
- Práctica: Nombres del negocio · Cómo la aplicamos: Solicitud, enviar, TransicionNoPermitida · Por qué ayuda a mantener: El código se lee como el proceso real
- Práctica: Pruebas y convenciones · Cómo la aplicamos: Una prueba por regla, ramas cortas y commits

con formato · Por qué ayuda a mantener: Cambiar sin miedo, con un historial legible

IDEAS CLAVE

- Una responsabilidad por clase
- Inyección por constructor
- Datos inmutables con records
- Errores en un solo lugar
- Nombres del negocio
- Pruebas y convenciones

QUÉ DECIR

"Un código mantenible es el que se puede cambiar sin miedo. Aplicamos seis prácticas sencillas."

"Cada clase tiene una sola responsabilidad: el controlador solo traduce HTTP, el servicio decide y el adaptador guarda. Las dependencias llegan por el constructor, así cualquier pieza se puede reemplazar o probar sola. Los datos son inmutables: cambiar de estado crea una solicitud nueva, en lugar de modificar la que había."

"Los errores se manejan en un solo lugar, los nombres son los del negocio, en español, y cada regla tiene su prueba. El resultado es que un cambio de regla toca un archivo, y una prueba nos dice si rompimos algo."

SI PREGUNTAN

¿Esto no hace el código más largo?
Un poco, pero cada pieza es corta y se entiende sola; el costo se paga solo en el primer cambio.

PARA PASAR

"Esto es lo que aprendimos de los fundamentos."

---

8. Fundamentos: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 67 pruebas en cada cambio
- Métrica: 0 bases de datos para probar
- El negocio no depende del framework: Las reglas son clases Java sin Spring, inyectadas por constructor. Valor: Se prueban en milisegundos y sobreviven a un cambio de framework.
- Los errores son parte del contrato: Un manejador global responde 400, 404 y 409 con la misma forma. Valor: Menos soporte: quien consume la API entiende qué pasó.
- Mantenible se diseña desde el inicio: Una responsabilidad por clase, inyección por constructor y records. Valor: Cambios pequeños y localizados.
- Todo entra por Pull Request: Ramas cortas, commits con sentido y una aprobación. Valor: Historial legible y ningún cambio sin revisar.

IDEAS CLAVE

- El negocio no depende del framework
- Los errores son parte del contrato
- Probar es barato si el diseño lo permite

QUÉ DECIR

“Cuatro aprendizajes de esta parte. Primero: el negocio no debe depender del framework. Mantener las reglas en clases Java simples nos permitió probarlas rápido y pensar en el negocio, no en las anotaciones.”

“Segundo: los errores son parte del contrato. Un buen 409 con un mensaje claro ahorra muchas horas de soporte.”

“Tercero: el código mantenible se diseña desde el inicio, con piezas pequeñas de una sola responsabilidad. Y cuarto: trabajar con ramas cortas y Pull Requests con aprobación nos dio un historial que cualquiera puede leer.”

PARA PASAR

“Con esas bases, veamos cómo está armado el sistema completo.”

2 · Arquitectura

Microservicios, puertos y adaptadores, y un contrato por eventos.

9. El sistema de un vistazo

EN PANTALLA · CAPAS DEL SISTEMA

- Experiencia: Persona: Usa el navegador; arka-lite-ui: React + nginx · proxy /api
- Servicios: servicio-solicitudes: El trámite y sus estados; Evento SolicitudEnviada: Por HTTP, con su eventoId; servicio-notificaciones: Los avisos, sin duplicados
- Datos: db-solicitudes: PostgreSQL 17 · solicitud y evento; db-notificaciones: PostgreSQL 17 · evento_procesado
- Plataforma: Compose o Kubernetes: Dónde corre; GitHub Actions: Prueba y publica; Loki y Grafana: Logs en un solo lugar

IDEAS CLAVE

- La UI es la puerta de entrada
- Dos servicios, cada uno dueño de su base
- Se hablan con un evento por HTTP
- La plataforma corre, entrega y observa

QUÉ DECIR

“Este es el sistema por capas. Arriba, la experiencia: la persona usa la interfaz, y la interfaz no guarda nada, solo llama a los servicios por un proxy.”

“En el medio, los dos microservicios. Solicitudes lleva el trámite; notificaciones crea los avisos. No comparten base: cada uno es dueño de sus datos, y se comunican con un evento por HTTP.”

“Y abajo, la plataforma que lo sostiene: dónde corre, cómo se prueba y publica cada cambio, y dónde vemos los logs. Cada capa tiene una sola responsabilidad, y por eso cada una se puede cambiar sin romper las otras.”

SI PREGUNTAN

¿Por qué no comparten una sola base de datos?

Para que cada servicio sea dueño de sus datos y pueda cambiar sin romper al otro.

PARA PASAR

"¿Y cómo se conectan esas piezas entre sí?"

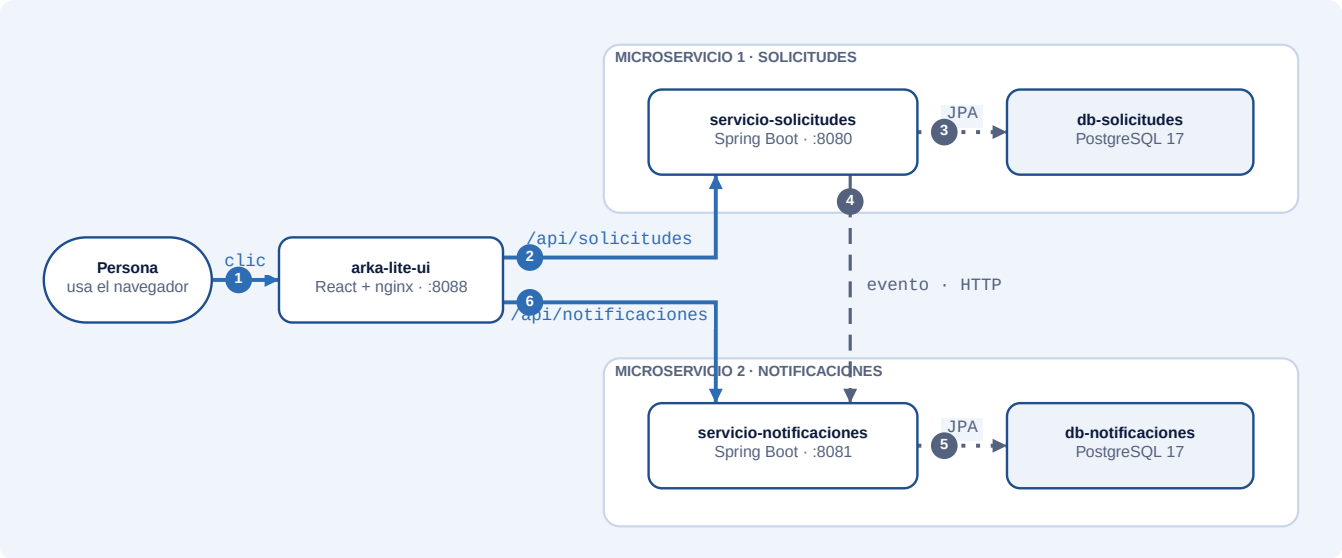
---

10. Cómo se conectan las piezas

EN PANTALLA

- UI -> solicitudes por /api
- Cada servicio -> su propia base
- solicitudes -> notificaciones con un evento
- UI -> notificaciones para ver los avisos

DIAGRAMA · EL MAPA DE ARKA-LITE



Petición · · · · Paso interno explicado — — Trabajo posterior

1. La persona usa la UI en el navegador.
2. La UI llama a solicitudes por /api: crear, listar, enviar, aprobar o rechazar.
3. Solicitudes guarda los datos en su propia base.
4. Al enviar, solicitudes avisa después a notificaciones con un evento por HTTP.
5. Notificaciones guarda en su base qué eventos ya consumió.
6. La UI también consulta a notificaciones: avisos y eventos consumidos.

Cada servicio es dueño de su base. Ninguno lee la base del otro.

QUÉ DECIR

“Sigamos los números. La persona usa la interfaz en su navegador. La interfaz llama a solicitudes por la ruta /api para crear, listar, enviar, aprobar o rechazar, y solicitudes guarda en su propia base.”

“Cuando una solicitud se envía, solicitudes le avisa después a notificaciones con un evento por HTTP, y notificaciones anota en su base qué eventos ya recibió. Por último, la interfaz también consulta a notificaciones para mostrar los avisos.”

“Fíjense en lo que no hay: ninguna flecha entre las dos bases. Los servicios solo se hablan por HTTP.”

PARA PASAR

“Veamos cada conexión con más detalle.”

---

11. Quién habla con quién, y cómo

EN PANTALLA · TABLA DE TRES COLUMNAS

- Conexión: Navegador -> UI · Cómo se habla: HTTP a localhost:8088; nginx sirve la página · Por qué así: El navegador nunca conoce las direcciones internas
- Conexión: UI -> servicios · Cómo se habla: nginx reenvía /api/solicitudes y /api/notificaciones · Por qué así: Cambiar una dirección no obliga a tocar la UI
- Conexión: solicitudes -> notificaciones · Cómo se habla: POST del evento SolicitudEnviada en JSON, desde el relevador · Por qué así: Si notificaciones cae, solicitudes sigue funcionando
- Conexión: Servicio -> su base · Cómo se habla: JDBC a db-solicitudes:5432 o db-notificaciones · Por qué así: Cada servicio es dueño de sus datos
- Conexión: Cómo se encuentran · Cómo se habla: Por nombre: la red de Compose o un Service de Kubernetes · Por qué así: Sin IPs fijas: funciona igual en cualquier ambiente
- Conexión: Seguimiento · Cómo se habla: El traceId viaja en la cabecera de cada llamada HTTP · Por qué así: Una sola búsqueda en Grafana sigue todo el recorrido

IDEAS CLAVE

- El navegador solo conoce la UI
- nginx reenvía /api al servicio
- El evento viaja después, desde el relevador
- Cada servicio habla solo con su base
- Se encuentran por nombre, no por IP
- El traceId acompaña cada llamada

QUÉ DECIR

“Esta tabla resume todas las conexiones. El navegador solo conoce la interfaz. La interfaz no llama directo a los servicios: nginx recibe todo lo que empieza con /api y lo reenvía al servicio que corresponde.”

“Entre servicios, solicitudes manda el evento a notificaciones con un POST en JSON, pero no en el momento de la petición: lo hace el relevador, después. Por eso, si notificaciones se cae, solicitudes sigue funcionando.”

“Cada servicio habla solo con su propia base. Y nadie usa direcciones IP: se encuentran por nombre, gracias a la red de Docker Compose o a los Services de Kubernetes. En cada llamada viaja además un traceId, que nos permite seguir el recorrido completo en los logs.”

SI PREGUNTAN

¿Por qué no se llaman directamente por localhost?
Porque dentro de un contenedor localhost es el mismo contenedor; por eso cada servicio se busca por su nombre.

PARA PASAR

“Hay dos formas de hablarse, y usamos las dos.”

---

12. Síncrono o asíncrono

EN PANTALLA · DOS MANERAS, LADO A LADO

- Síncrono: pido y espero: La UI llama y espera la respuesta; Crear, listar, aprobar o rechazar; Respuesta inmediata: 201, 404, 409; Si el servicio está caído, la persona ve el error
- Asíncrono: aviso después: Solicitudes guarda el evento y responde ya; El relevador lo entrega cada 5 segundos; Si notificaciones cae, el evento espera; Mismo contrato JSON: SolicitudEnviada

IDEAS CLAVE

- Síncrono: pido y espero la respuesta
- Asíncrono: guardo, respondo y aviso después
- La UI usa síncrono; el evento, asíncrono
- Lo asíncrono aguanta que el otro esté caído

QUÉ DECIR

“Hay dos formas de comunicarse. Síncrona: pido y espero la respuesta, como una llamada telefónica. Es lo que hace la interfaz al crear o aprobar una solicitud, porque la persona necesita saber el resultado en ese momento.”

“Asíncrona: dejo el mensaje y sigo, como un correo. Es lo que hace solicitudes con notificaciones: guarda el evento, responde enseguida y el relevador lo entrega después.”

“La regla que aprendimos: lo que la persona necesita ver, síncrono; lo que puede esperar, asíncrono. Así una caída de notificaciones no afecta a quien está usando solicitudes.”

PARA PASAR

“Ahora entremos a uno de los servicios.”

---

13. Por dentro: puertos y adaptadores

EN PANTALLA · PUERTOS Y ADAPTADORES

- Centro: Dominio: Solicitud · ServicioDeSolicitudes
- Puertos: RepositorioDeSolicitudes, RegistradorDeEventosSalientes, PublicadorDeEventos
- Entrada: SolicitudController: REST · Swagger; Manejador global: Errores -> HTTP
- Salida: RepositorioJpa: @Primary · PostgreSQL; Bandeja de salida: Tabla evento; PublicadorConReintento: @Primary · HTTP; RepositorioEnMemoria: Para las pruebas

IDEAS CLAVE

- Entrada: traduce HTTP al dominio
- Dominio: las reglas, sin Spring
- Puertos: contratos que el dominio necesita
- Salida: PostgreSQL y HTTP, intercambiables

Comparación: Como un enchufe: el aparato no sabe de qué planta viene la electricidad, solo que el enchufe cumple la norma.

QUÉ DECIR

“¿Qué quiere decir hexagonal? El nombre viene del dibujo: un hexágono con el negocio al centro y cada lado como un enchufe hacia afuera. También se le llama puertos y adaptadores. El dominio no llama a la base: pide lo que necesita a través de puertos, que son interfaces.”

“Para quien trabaja con procedimientos almacenados es una idea conocida: la aplicación llama a un procedimiento por su nombre y sus parámetros, sin saber cómo está escrito por dentro. Aquí es al revés: el negocio declara qué necesita, y la tecnología se enchufa a ese contrato.”

“A la izquierda están los adaptadores de entrada, como el controlador REST. A la derecha, los de salida: el que guarda en PostgreSQL, el que anota el evento pendiente y el que lo publica con reintentos.”

“Fíjense en el de abajo: un repositorio en memoria que cumple el mismo puerto. Lo usan las pruebas, y Spring elige el que usa la aplicación real con @Primary. Por eso cambiar la base o el mecanismo de entrega no obliga a tocar las reglas del negocio.”

SI PREGUNTAN

¿Esto no es más código que una aplicación en capas tradicional?

Un poco más de interfaces, a cambio de probar sin base y cambiar adaptadores sin tocar reglas.

PARA PASAR

“¿Y por qué dos servicios y no uno solo?”

14. Arquitectura: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 3 puertos en solicitudes
- Métrica: 2 adaptadores por puerto
- Los puertos separan negocio y tecnología: Varios adaptadores por puerto; Spring usa el @Primary. Valor: Cambiar la base o la mensajería sin reescribir reglas.
- Separar servicios tiene un costo: Red, bases propias y fallos parciales. Valor: Separar solo cuando el negocio o los equipos lo justifican.
- Síncrono para ver, asíncrono para avisar: La UI espera la respuesta; el evento se entrega después. Valor: Una caída parcial no detiene a quien usa el sistema.
- El contrato es lo único compartido: SolicitudEnviada vive en los dos servicios y cambia junto. Valor: Equipos independientes con un acuerdo explícito.

IDEAS CLAVE

- Los puertos separan negocio y tecnología
- Separar servicios tiene un costo
- El contrato entre servicios se versiona con cuidado

QUÉ DECIR

"Primero: los puertos separan el negocio de la tecnología. Hoy usamos PostgreSQL y HTTP, pero cambiar a otra base o a un sistema de mensajería no obligaría a reescribir las reglas."

"Segundo: separar servicios tiene un costo real. Lo vale cuando el negocio o los equipos lo piden, no antes."

"Tercero: síncrono para lo que la persona necesita ver y asíncrono para lo que puede esperar. Y cuarto: entre dos servicios, lo único compartido debe ser el contrato. Si cambia, cambia en los dos lados, de forma coordinada."

PARA PASAR

"El costo más interesante de separar está en los datos. Vamos a la base de datos."

3 · Base de datos

Una base por servicio, un esquema que se cuida solo y datos consistentes.

15. Del procedimiento almacenado al ORM

EN PANTALLA · DOS MANERAS, LADO A LADO

- Con procedimientos almacenados: La regla vive en la base: EXEC sp_enviar; Cambiar una regla es cambiar la base; Para probar hace falta la base con datos; Atado a un motor: T-SQL o PL/SQL
- Con un ORM: JPA + Hibernate: La regla vive en Java, en el dominio; Hibernate genera el SQL de guardar y buscar; Las reglas se prueban sin base, en milisegundos; Cambiar de motor no toca las reglas

IDEAS CLAVE

- Con SP, la regla vive en la base
- Con ORM, la regla vive en el código
- Hibernate escribe el INSERT, UPDATE y SELECT
- No es todo o nada: el SQL pesado puede seguir en la base

QUÉ DECIR

"Muchos de ustedes trabajan con procedimientos almacenados: la regla del negocio vive en la base, y la aplicación solo hace EXEC con parámetros. Funciona, pero cambiar una regla es cambiar la base, y probarla exige una base con datos."

"En ARKA-Lite usamos un ORM, que significa mapeo objeto-relacional. En Java el estándar se llama JPA y la herramienta que lo hace es Hibernate. Una clase Java se marca como entidad y queda asociada a una tabla: cada campo es una columna. Cuando guardamos o buscamos, Hibernate escribe el INSERT, el UPDATE o el SELECT por nosotros."

"Ojo: no es todo o nada. Las reglas del negocio pasan al código, donde se prueban en milisegundos, y lo que de verdad es trabajo de base de datos, como reportes pesados, puede seguir en SQL."

SI PREGUNTAN

¿Y el rendimiento? ¿El SQL que genera Hibernate es bueno?

Para guardar y buscar por llave, sí. Para consultas complejas se puede escribir SQL propio; además, los índices y constraints siguen siendo trabajo de la base.

PARA PASAR

"¿Y cómo llega, paso a paso, una línea de Java hasta PostgreSQL?"

---

16. Cómo el código llega a la base

EN PANTALLA · FLUJO PASO A PASO

- 1. Dominio: Pide guardar la solicitud, sin SQL.
- 2. Spring Data: save() y findById(), sin escribir SQL.
- 3. Hibernate: Objeto -> INSERT o UPDATE.
- 4. JDBC + pool: Reutiliza conexiones con usuario y clave.
- 5. PostgreSQL: Ejecuta y confirma la transacción.

IDEAS CLAVE

- El dominio pide guardar, sin SQL
- Spring Data ofrece save y findById
- Hibernate convierte el objeto en SQL
- JDBC y un pool de conexiones hablan con PostgreSQL

Comparación: Como un traductor: el código habla Java, la base habla SQL, y el ORM traduce en los dos sentidos.

QUÉ DECIR

“Sigamos una línea de código. El dominio dice: guardar esta solicitud. No sabe nada de SQL. Spring Data nos da métodos listos, como save o findById, sin escribir una sola consulta.”

“Por debajo, Hibernate convierte el objeto en un INSERT o un UPDATE. Y el driver JDBC, con un pool de conexiones que las reutiliza, como el que ya conocen de cualquier aplicación, se lo manda a PostgreSQL, que ejecuta y confirma la transacción.”

“La dirección de la base, el usuario y la clave no están en el código: llegan por variables de entorno.”

PARA PASAR

“¿Y dónde se configura todo eso?”

---

17. Dónde se configura la conexión

EN PANTALLA · TARJETAS

- URL, usuario y clave: spring.datasource lee DB_URL, DB_USER y DB_PASSWORD; nada fijo en el código.
- Secretos fuera de la imagen: En Compose, variables de entorno; en Kubernetes, el Secret arka-postgres.
- Pool de conexiones: HikariCP viene con Spring Boot: abre unas pocas conexiones y las reutiliza.
- Transacciones: @Transactional es el BEGIN … COMMIT: si algo falla, ROLLBACK de todo.
- Arranque seguro: schema.sql crea lo que falta y Hibernate valida que las tablas coincidan.
- Por nombre: jdbc:postgresql://db-solicitudes:5432 dentro de la red; nunca localhost.

IDEAS CLAVE

- URL, usuario y clave por variables
- Secretos fuera de la imagen
- Un pool que reutiliza conexiones
- Transacciones todo o nada
- Tablas listas y validadas al arrancar
- La base se busca por nombre

QUÉ DECIR

“Para quienes administran bases de datos, esta es la parte conocida. La aplicación no trae la dirección de la base ni la clave: las lee de variables de entorno. En Docker Compose las pone el archivo compose; en Kubernetes, la clave sale de un Secret.”

“Spring Boot incluye un pool de conexiones, HikariCP, que abre unas pocas conexiones y las reutiliza, igual que cualquier aplicación bien configurada. Y la anotación @Transactional es el BEGIN y COMMIT de siempre: si algo falla en el medio, se hace ROLLBACK de todo.”

“Al arrancar, el servicio crea las tablas que falten con schema.sql y Hibernate valida que coincidan con el código. Y dentro de la red, la base se busca por su nombre, db-solicitudes, nunca por localhost.”

SI PREGUNTAN

¿Cuántas conexiones abre cada servicio?

Las que define el pool; por defecto HikariCP mantiene hasta 10 y las reutiliza entre peticiones.

PARA PASAR

"Ahora veamos qué tablas tiene cada servicio."

---

18. Una base por servicio

EN PANTALLA · BASES DE DATOS Y SUS TABLAS

- db-solicitudes (PostgreSQL 17 · schema.sql): solicitud [id · PK, tipo, estado · CHECK (4 estados)]; evento [solicitud_id · FK, evento_id · UUID, procesado · false = pendiente]
- db-notificaciones (PostgreSQL 17 · Hibernate): evento_procesado [evento_id · PK, un registro por evento recibido]
- Entre bases: evento_id por HTTP

IDEAS CLAVE

- PostgreSQL 17 en cada servicio
- solicitud y evento viven juntas
- evento_procesado recuerda lo recibido
- Ninguna llave foránea cruza bases

QUÉ DECIR

"Cada servicio tiene su propia base PostgreSQL. En la de solicitudes hay dos tablas: la solicitud, con un CHECK que solo admite los cuatro estados válidos, y la tabla evento, que funciona como bandeja de salida: cada fila es un aviso pendiente de entregar."

"En la de notificaciones hay una sola tabla importante: evento_procesado, la lista de eventos que ya recibió."

"Fíjense en la flecha: no hay llaves foráneas entre bases. Lo único que viaja de un lado al otro es el evento_id. Esa es la frontera entre los dos servicios."

SI PREGUNTAN

¿Por qué no un JOIN entre las dos bases?

Porque cada servicio es dueño de sus datos; un JOIN entre bases los volvería a acoplar.

PARA PASAR

"Ahora, el problema más interesante: ¿qué pasa si guardo y el otro servicio está caído?"

---

19. Guardar primero, avisar después

EN PANTALLA · FLUJO PASO A PASO

- 1. Enviar: Solicitud ENVIADA + evento pendiente, en una transacción.
- 2. Bandeja de salida: El evento espera en la tabla evento.
- 3. Relevador: Cada 5 s entrega los pendientes, hasta 3 intentos.
- 4. Notificaciones: Lo recibe y el evento queda procesado.
- Si si falla en el paso 3: Sigue pendiente: Mismo eventoId; se reintenta en el siguiente ciclo.

IDEAS CLAVE

- Solicitud ENVIADA y evento pendiente, juntos
- Si una falla, no se guarda ninguna
- El relevador entrega cada 5 s, con 3 intentos
- Si falla, sigue pendiente con el mismo eventoId

Comparación: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

QUÉ DECIR

"Este es el problema de la consistencia. Si al enviar una solicitud llamáramos directamente a notificaciones y justo estuviera caído, la solicitud quedaría enviada y nadie se enteraría."

"La solución se llama patrón outbox, o bandeja de salida. Al enviar, guardamos la solicitud y un evento pendiente en la misma transacción: o se guardan las dos cosas, o ninguna. La respuesta le llega a la persona enseguida."

"Después, un relevador revisa la bandeja cada cinco segundos y entrega los pendientes, con hasta tres intentos. Si aun así falla, el evento sigue pendiente, con el mismo número, para el siguiente ciclo."

SI PREGUNTAN

¿Que el envío responda bien significa que el aviso ya existe?
No. Significa que la solicitud y el evento quedaron guardados; el aviso llega después.

PARA PASAR

"Si se reintenta, ¿no puede llegar el aviso dos veces?"

---

20. El mismo evento, un solo aviso

EN PANTALLA · FLUJO PASO A PASO

- 1. Llega el evento: Trae su eventoId.
- 2. ¿Ya lo recibimos?: Se busca en evento_procesado.
- 3. Crear el aviso: Solo si el número es nuevo.
- 4. Anotar el número: En la misma transacción.
- Si ya estaba en el paso 2: Ignorarlo: No se crea otro aviso. Duplicados: cero.

IDEAS CLAVE

- Cada evento trae un número que no cambia
- Notificaciones anota los números recibidos
- Si llega uno repetido, lo ignora
- Se comprueba con el botón Reentregar

Comparación: Como un portero con lista de invitados: si alguien ya entró, no lo vuelve a anotar.

QUÉ DECIR

“Sí, puede llegar dos veces. Si notificaciones recibió el evento pero su respuesta se perdió, el relevador cree que falló y lo vuelve a mandar.”

“Para eso está la idempotencia: repetir la misma operación no multiplica su efecto. Notificaciones funciona como un portero con lista: anota el número de cada evento, y si llega uno que ya está en la lista, lo ignora.”

“El outbox evita perder avisos; la idempotencia evita duplicarlos. Juntos dan lo que en la industria se llama entrega al menos una vez con efecto exactamente una vez.”

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, pestaña Enviadas: Reentregar un evento ya consumido. Debe verse: El diálogo “El aviso no se duplicó” y Avisos duplicados en 0.

SI PREGUNTAN

¿Qué resuelve el outbox y qué resuelve la idempotencia?
El outbox evita perder el evento; la idempotencia evita duplicar su efecto.

PARA PASAR

“Esto es lo que aprendimos de la base de datos.”

---

21. Base de datos: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 0 avisos perdidos
- Métrica: 0 avisos duplicados
- El ORM no elimina el trabajo de base de datos: Hibernate escribe el SQL rutinario; índices, CHECK y transacciones siguen siendo diseño de BD. Valor: Reglas que se prueban en código y una base que se cuida sola.
- Guardar primero, avisar después: Solicitud y evento en una transacción local; el relevador entrega después. Valor: Un servicio caído no hace perder información.
- Todo reintento necesita idempotencia: Un eventoId fijo y una tabla de eventos ya procesados. Valor: Se puede reintentar sin miedo a duplicar efectos.
- El esquema también es código: schema.sql idempotente, CHECK, índice parcial y validación al arrancar. Valor: Despliegues repetibles y errores visibles al inicio.

IDEAS CLAVE

- La transacción local es la herramienta más confiable
- Reintentar exige idempotencia
- El esquema también es código

QUÉ DECIR

“Cuatro aprendizajes de datos. El primero, para quienes vienen de los procedimientos almacenados: el ORM no elimina el trabajo de base de datos. Escribe el SQL rutinario, pero los índices, los CHECK y las transacciones siguen siendo diseño de base.”

“Segundo: la transacción local es la herramienta más confiable que tenemos. En lugar de coordinar dos servicios al mismo tiempo, guardamos todo en nuestra base y avisamos después.”

“Tercero: cualquier reintento necesita idempotencia del otro lado. Si no, los reintentos que nos protegen de perder datos nos terminan duplicando datos.”

“Y cuarto: el esquema también es código. Versionado, idempotente y validado al arrancar.”

PARA PASAR

“Los datos están cuidados. Ahora, ¿cómo llevamos todo esto a cualquier ambiente?”

4 · Contenedores y Kubernetes

El mismo sistema en cualquier computadora y un clúster que se repara solo.

22. ¿Qué es Docker y qué es Kubernetes?

EN PANTALLA · DOS MANERAS, LADO A LADO

- Docker: la caja: Empaqueta la app con Java y todo lo que usa; Imagen: un paquete que no cambia; Corre igual en una laptop y en un servidor; Compose levanta varias cajas juntas
- Kubernetes: el administrador: Decide dónde corre cada caja; Si una se cae, levanta otra; Le da a cada servicio un nombre fijo; Puede correr más copias si hay carga

IDEAS CLAVE

- Docker: la caja con la app y todo lo que necesita
- Imagen: el paquete; contenedor: el paquete corriendo
- Kubernetes: arranca, vigila y reinicia las cajas
- Compose hace lo mismo en una sola computadora

Comparación: Como el transporte de carga: Docker es el contenedor estándar y Kubernetes, el puerto que los acomoda, los vigila y reemplaza el que se daña.

QUÉ DECIR

“Dos palabras que se escuchan mucho. Docker resuelve el clásico “en mi máquina funciona”: empaqueta la aplicación con Java y todo lo que necesita en una imagen, un paquete que no cambia. Cuando esa imagen corre, se llama contenedor, y corre igual en una laptop o en un servidor.”

“Kubernetes es el siguiente nivel: no empaqueta, administra. Decide dónde corre cada contenedor, lo vigila, y si uno se cae, levanta otro. Piénsenlo como un puerto de carga: Docker es el contenedor estándar y Kubernetes es el puerto que los acomoda y reemplaza el que se daña.”

“En una sola computadora usamos Docker Compose; en un clúster, Kubernetes. La misma imagen sirve en los dos.”

SI PREGUNTAN

¿Docker es una máquina virtual?

No. Es más liviano: comparte el sistema operativo y solo aísla la aplicación y lo que necesita.

PARA PASAR

“Veamos el sistema completo levantado con Docker.”

---

23. Todo el sistema con un comando

EN PANTALLA · TARJETAS

- Comando: docker compose up --build -d
- arka-lite-ui: :8088 · la interfaz y su proxy
- servicio-solicitudes: :8080 · el trámite
- servicio-notificaciones: :8081 · los avisos
- db-solicitudes: PostgreSQL 17 · solo red interna
- db-notificaciones: PostgreSQL 17 · :5433
- promtail: recoge los logs
- loki: :3100 · guarda los logs
- grafana: :3000 · busca en los logs

IDEAS CLAVE

- Servicios, bases, UI y logs en un archivo
- Cada servicio se llama por su nombre
- La configuración entra por variables
- La misma imagen sirve en Compose y en Kubernetes

QUÉ DECIR

“¿Cuántas veces hemos escuchado “en mi máquina funciona”? Con Docker, cada servicio viaja con su receta, y con Compose levantamos el sistema completo con un solo comando: los dos servicios, sus bases, la interfaz y las herramientas de logs.”

“Dentro de la red de Compose, cada servicio encuentra al otro por su nombre, no por localhost. Y la configuración entra por variables de entorno, así que la misma imagen sirve en nuestra computadora y en el clúster, sin recompilar.”

“Para un equipo nuevo, esto es la diferencia entre un día configurando y cinco minutos.”

QUÉ HACER (NO SE DICE)

- Mostrar: docker compose ps Debe verse: La UI, los servicios y las bases arriba y sanos.

SI PREGUNTAN

Dentro de un contenedor, ¿a dónde lleva localhost?
Al mismo contenedor, no a otro servicio. Por eso usamos el nombre del servicio.

PARA PASAR

“En nuestra computadora usamos Compose. En un clúster, Kubernetes.”

---

24. Cuando un Pod se cae

EN PANTALLA · QUÉ PASA CUANDO FALLA

- Falla: Se cae el Pod de solicitudes: Un error, un nodo reiniciado o un kubectl delete.
- 1. [0 s] Detecta: Falta 1 de 1 copias.
- 2. [+1 s] Recrea: Un Pod nuevo con la misma imagen.
- 3. [+segundos] Espera: initContainers: base y notificaciones.
- 4. [Ready] Listo: La readinessProbe responde.
- Resultado: Sin intervención manual: Los eventos pendientes siguen en la base y se entregan.

IDEAS CLAVE

- El Deployment detecta que falta una copia
- Crea un Pod nuevo
- Espera a la base y a su salud
- Vuelve a recibir tráfico, sin intervención

QUÉ DECIR

“Kubernetes funciona como un termostato: no le decimos qué hacer paso a paso, le decimos el estado que queremos, por ejemplo una copia de cada servicio, y él lo mantiene.”

“Si el Pod de solicitudes se cae, el Deployment detecta que falta una copia y crea otro con la misma imagen. El Pod nuevo espera a que su base y notificaciones respondan, y solo cuando su prueba de salud contesta, el Service le vuelve a mandar tráfico.”

“Y como los eventos pendientes están en la base, no en memoria, nada se pierde mientras tanto. Nadie tuvo que despertarse a las tres de la mañana.”

QUÉ HACER (NO SE DICE)

- Mostrar: kubectl delete pod -l app=servicio-solicitudes y luego kubectl get pods -w. Debe verse: Aparece un Pod nuevo que vuelve a estar Ready.

SI PREGUNTAN

¿Y si se cae la base?

Su Pod también se recrea y los datos siguen en el disco persistente; los servicios esperan a que vuelva.

PARA PASAR

"Esto es lo que aprendimos de la plataforma."

---

25. Contenedores y Kubernetes: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 1 comando para levantar todo
- Métrica: 5 manifiestos en k8s/
- La imagen es el entregable: Dos etapas, solo Java y el JAR, usuario sin privilegios. Valor: Lo que se prueba es exactamente lo que se despliega.
- Configurar por variables: La misma imagen en Compose y en Kubernetes. Valor: Un solo artefacto para todos los ambientes.
- Declarar el estado deseado: Deployments, initContainers y readinessProbe. Valor: El clúster se repara solo y no recibe tráfico antes de tiempo.

IDEAS CLAVE

- La imagen es el entregable
- Configurar por variables, no por código
- Declarar el estado y dejar que se mantenga

QUÉ DECIR

"Primero: la imagen es el entregable. Lo que se prueba es exactamente lo que se despliega."

"Segundo: configurar por variables de entorno y no por código. Así hay un solo artefacto para todos los ambientes."

"Y tercero: con Kubernetes declaramos el estado que queremos y el clúster lo mantiene. Arrancar no es lo mismo que estar listo, y eso hay que decírselo explícitamente."

PARA PASAR

"¿Y quién construye y prueba esas imágenes? Nadie a mano: el pipeline."

5 · CI/CD

Integración y entrega continua: cada cambio se prueba y se empaqueta solo.

26. Cada cambio se prueba solo

EN PANTALLA · LAS ETAPAS DEL PIPELINE

- 1. Push o PR: Alguien sube un cambio.
- 2. Compilar: Java 26 y Maven, un job por servicio.
- 3. Probar: mvn verify con PostgreSQL de servicio.
- 4. Arrancar: Levanta el JAR y espera ver CAM-002.
- 5. Imagen: Se construye con Docker.
- 6. Publicar: En GHCR, con el SHA del commit.

IDEAS CLAVE

- Cada push o Pull Request dispara el pipeline
- Compila y prueba cada servicio con su base
- Comprueba que el servicio realmente arranca
- Publica la imagen con el SHA del commit

QUÉ DECIR

"Con GitHub Actions, cada vez que alguien sube un cambio o abre un Pull Request, se dispara el pipeline. Por cada servicio prepara Java y una base PostgreSQL, compila y corre todas las pruebas."

"Luego hace algo que muchos pipelines no hacen: arranca el servicio de verdad y espera a ver los datos iniciales. Así detectamos errores de configuración que las pruebas unitarias no ven."

"Si todo pasa, construye la imagen y la publica etiquetada con el código del commit. Cada imagen se puede rastrear hasta el cambio exacto que la produjo."

SI PREGUNTAN

¿Publicar la imagen la despliega en Kubernetes?

No. Publicar deja la imagen lista; desplegar es otro paso, con kubectl apply.

PARA PASAR

"El pipeline funciona porque todo entra por el mismo camino."

27. CI/CD: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 2 jobs en paralelo, uno por servicio
- Métrica: 1 imagen por commit
- Probar que arranca, no solo que compila: El pipeline levanta el JAR y consulta GET /solicitudes. Valor: Los errores de configuración aparecen antes de desplegar.
- Cada imagen se rastrea a su commit: Etiqueta con el SHA en GHCR. Valor: Saber qué código corre y volver atrás con certeza.
- El pipeline es la puerta: Pull Request, una aprobación y todo en verde. Valor: La calidad no depende de la memoria de nadie.

IDEAS CLAVE

- Probar que arranca, no solo que compila

- Cada imagen se rastrea a su commit
- El pipeline es la puerta de entrada

QUÉ DECIR

“Primero: hay que probar que el servicio arranca, no solo que compila. Muchos errores de configuración solo aparecen al arrancar.”

“Segundo: cada imagen se etiqueta con su commit. Eso nos dice exactamente qué código está corriendo y nos permite volver atrás con certeza.”

“Y tercero: el pipeline es la puerta de entrada. La calidad no depende de que alguien se acuerde de correr las pruebas.”

PARA PASAR

“Ya está desplegado. ¿Cómo sabemos qué pasa adentro?”

6 · Observabilidad

Logs centralizados, trazas entre servicios y salud de cada componente.

28. Todos los logs, en un solo lugar

EN PANTALLA · FLUJO PASO A PASO

- 1. Servicios: Logs JSON con traceId y eventoId.
- 2. Promtail: Los recoge de cada contenedor.
- 3. Loki: Los guarda, etiquetados por servicio.
- 4. Grafana: Una búsqueda para todo el sistema.

IDEAS CLAVE

- Cada servicio escribe sus logs en JSON (ECS)
- Promtail los recoge de todos los contenedores
- Loki los guarda por servicio
- Grafana los busca por traceId o eventoId

Comparación: Como la caja negra de un avión: registra lo que pasó, para revisarlo después.

QUÉ DECIR

“Con dos servicios, cada uno tiene sus propios logs. Si algo falla, no queremos entrar contenedor por contenedor a buscar.”

“Cada servicio escribe sus logs en JSON, con un formato estándar. Promtail los recoge de todos los contenedores, Loki los guarda etiquetados por servicio y Grafana nos deja buscarlos en un solo lugar.”

“Las llamadas entre servicios llevan el traceId, así que con una sola búsqueda vemos el viaje de un evento: cuándo lo publicó el relevador y cuándo lo consumió notificaciones.”

QUÉ HACER (NO SE DICE)

- Mostrar: Grafana ? Explore ? Loki: {service="servicio-notificaciones"} |= "evento". Debe verse: Las líneas “Consume evento” y “aviso creado” con el número del evento.

PARA PASAR

“Los logs cuentan qué pasó. La salud dice cómo estamos ahora.”

---

29. Salud de cada servicio

EN PANTALLA · CAPTURAS REALES

- Captura de La pestaña Salud de la UI (localhost:8088/#/salud)

IDEAS CLAVE

- Actuator expone health, info y métricas
- La salud revisa la conexión con la base
- Kubernetes y la UI la consultan
- UP no garantiza que todos los avisos llegaron

QUÉ DECIR

“Cada servicio expone su salud con Spring Boot Actuator: por dentro revisa, por ejemplo, que pueda hablar con su base. La interfaz la muestra en la pestaña Salud.”

“Pero ojo: que un servicio diga UP solo significa que responde. No garantiza que todos los avisos llegaron. Para eso miramos los eventos pendientes y los logs.”

SI PREGUNTAN

Si todo está UP y falta un aviso, ¿qué miramos?

Si el evento sigue pendiente en Enviadas, y los logs del relevador y de notificaciones.

PARA PASAR

“Esto es lo que aprendimos de la observabilidad.”

---

30. Observabilidad: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 1 lugar para buscar todos los logs
- Métrica: 8 contenedores observados
- Logs estructurados desde el día uno: JSON con formato ECS, sin cambiar el código de negocio. Valor: Buscar por campo en lugar de leer texto.
- Un identificador que cruce servicios: traceId propagado por el RestClient y eventoId en cada log. Valor: Diagnosticar un fallo entre servicios en minutos.
- Salud no es lo mismo que correcto: UP dice que responde; los pendientes dicen si entrega. Valor: Medir el resultado del negocio, no solo el proceso.

IDEAS CLAVE

- Logs estructurados desde el día uno
- Un identificador que cruce servicios
- Salud no es lo mismo que correcto

QUÉ DECIR

“Primero: los logs estructurados se configuran desde el día uno. Después es mucho más caro.”

“Segundo: en un sistema distribuido hace falta un identificador que cruce los servicios. Sin el traceId o el eventoId, diagnosticar es adivinar.”

“Y tercero: salud no es lo mismo que correcto. Un servicio puede estar UP y tener avisos pendientes; hay que medir el resultado, no solo el proceso.”

PARA PASAR

“Todo esto también tiene que estar documentado.”

7 · Documentación

Contratos que salen del código y documentación que cambia con él.

31. Contratos vivos con Swagger

EN PANTALLA · CAPTURAS REALES

- Captura de Swagger de solicitudes (localhost:8080/swagger-ui/index.html)

IDEAS CLAVE

- Cada servicio publica su contrato OpenAPI
- Swagger UI lo muestra y permite probarlo
- Un endpoint nuevo aparece solo
- La UI enlaza los dos Swagger

Comparación: Como el menú de un restaurante: dice qué hay, qué lleva cada plato y qué te van a traer.

QUÉ DECIR

“Un contrato de API es como el menú de un restaurante: dice qué se puede pedir, qué lleva cada pedido y qué te van a devolver.”

“Cada servicio publica su contrato con el estándar OpenAPI, usando springdoc, y Swagger lo muestra como una página donde se puede ver cada endpoint, sus códigos de respuesta y probarlo en vivo.”

“Lo importante para el staff: se genera desde el código. Cuando agregamos un endpoint, aparece solo, y nunca queda desactualizado. Un equipo que consume la API no tiene que pedirnos un documento.”

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, pestaña Salud ? Swagger · solicitudes, y ejecutar GET /solicitudes. Debe verse: La lista en JSON con código 200.

PARA PASAR

“Y lo que no sale del código, lo documentamos junto al código.”

32. Documentación: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 11 endpoints documentados solos
- Métrica: 2 Swagger, uno por servicio
- Lo que se genera no se desactualiza: OpenAPI sale de los controladores con springdoc. Valor: Cero horas manteniendo un documento de API aparte.
- La doc se revisa en el mismo PR: El cambio y su explicación llegan juntos. Valor: Quien revisa entiende el porqué, no solo el qué.
- Documentar para quien llega mañana: Guías en PowerShell, diagramas y la Ayuda de la UI. Valor: Menos tiempo para que alguien nuevo aporte.

IDEAS CLAVE

- Lo que se genera no se desactualiza
- La doc se revisa en el mismo PR
- Documentar para quien llega mañana

QUÉ DECIR

"Primero: lo que se genera desde el código no se desactualiza. Es la mejor documentación de API que existe."

"Segundo: la documentación se revisa en el mismo Pull Request que el cambio."

"Y tercero: documentamos para la persona que llega mañana al equipo. Ese es el verdadero usuario de la documentación."

PARA PASAR

"Para ver todo esto funcionando, construimos una interfaz."

8 · La UI como laboratorio

No es parte del entregable: es el banco de pruebas para ver cómo se comporta el sistema, sobre todo cuando algo falla.

33. Una UI para ver el sistema por dentro

EN PANTALLA · CAPTURAS REALES

- Captura de Solicitudes (localhost:8088/#/solicitudes): Crear, enviar y decidir.
- Captura de Enviadas (localhost:8088/#/enviadas): Pendientes, entregados y Reentregar.
- Captura de Salud (localhost:8088/#/salud): UP o DOWN, Swagger y Grafana.
- Captura de Ayuda (localhost:8088/#/ayuda): Diagramas paso a paso.

IDEAS CLAVE

- Solicitudes: crear, enviar, aprobar o rechazar
- Enviadas: eventos pendientes, entregados y Reentregar
- Salud: los dos servicios, Swagger y Grafana
- Ayuda: diagramas de cómo funciona

QUÉ DECIR

“Una aclaración importante: esta interfaz no es parte del entregable. Los entregables son los dos microservicios y su plataforma. La interfaz la agregamos como laboratorio, para que cualquiera pueda ver lo que normalmente es invisible.”

“En Solicitudes se crean y se envían. En Enviadas se ve la bandeja de salida: qué eventos están pendientes, cuáles se entregaron, y un botón para reentregar a propósito. En Salud, si cada servicio está UP o DOWN. Y en Ayuda, diagramas de cómo funciona.”

“Donde más vale es cuando algo falla. Veamos eso.”

PARA PASAR

“Primer caso: el servicio de notificaciones se cae.”

---

34. Si notificaciones se cae

EN PANTALLA · QUÉ PASA CUANDO FALLA

- Falla: notificaciones está caído: docker compose stop servicio-notificaciones
- 1. [0 s] Enviar: Responde 200: solicitud ENVIADA y evento guardado.
- 2. [cada 5 s] Reintenta: 3 intentos por ciclo; el evento sigue pendiente.
- 3. [en la UI] Se ve: “Notificaciones no responde” y 1 en camino.
- 4. [al volver] Entrega: Mismo eventoId: se crea el aviso.
- Resultado: 1 aviso, 0 perdidos: Sin intervención y sin duplicados.

IDEAS CLAVE

- Enviar responde bien aunque notificaciones esté caído
- El evento queda pendiente en la bandeja
- El relevador reintenta cada 5 segundos
- Al volver, llega un solo aviso

QUÉ DECIR

“Detenemos notificaciones a propósito. En la pestaña Salud aparece en DOWN.”

“Ahora enviamos una solicitud. La respuesta es buena: la solicitud queda enviada y su evento queda guardado. La persona no ve ningún error, porque no depende de que el otro servicio esté arriba.”

“En Enviadas aparece un aviso: notificaciones no responde y el evento espera en la bandeja de salida. El relevador lo intenta cada cinco segundos y falla, pero no lo pierde. Cuando levantamos notificaciones, en el siguiente ciclo lo entrega con el mismo número, y aparece exactamente un aviso.”

QUÉ HACER (NO SE DICE)

- Antes: docker compose stop servicio-notificaciones. Mostrar Salud en DOWN.
- Crear y enviar DEMO-01; mostrar en Enviadas el aviso amarillo “Notificaciones no responde” y “1 en camino”. No hace falta recargar: la pantalla se actualiza sola y el indicador de servicios vuelve a 2 de 2.
- docker compose start servicio-notificaciones y esperar unos segundos: el evento pasa a entregado y aparece 1 aviso.

- Mostrar: Detener notificaciones, enviar DEMO-01, mirar Enviadas y volver a iniciar notificaciones. Debe verse: Evento en camino mientras está caído; al volver, entregado y 1 aviso.

SI PREGUNTAN

¿Cuánto tiempo puede esperar un evento pendiente?
Indefinidamente: está en la base. Se entrega en el primer ciclo después de que notificaciones vuelve.

PARA PASAR

"Ese es el caso más importante, pero no el único."

---

35. Cómo se comporta cuando algo falla

EN PANTALLA · TABLA DE TRES COLUMNAS

- Qué falla: notificaciones está caído · Cómo reacciona el sistema: El evento queda pendiente y se reintenta cada 5 s · Qué se ve en la UI: Aviso "Notificaciones no responde" · 1 en camino
- Qué falla: El evento llega dos veces · Cómo reacciona el sistema: ConsumoIdempotente lo reconoce y lo ignora · Qué se ve en la UI: "El aviso no se duplicó" · 0 duplicados
- Qué falla: Enviar dos veces o aprobar un borrador · Cómo reacciona el sistema: El dominio rechaza la transición: 409 · Qué se ve en la UI: Diálogo con el motivo del 409
- Qué falla: Faltan datos o el id ya existe · Cómo reacciona el sistema: Se valida antes de guardar: 400 o 409 · Qué se ve en la UI: Diálogo "Faltan datos" o "Ese identificador ya existe"
- Qué falla: Un POST tarda o se corta · Cómo reacciona el sistema: La UI no lo repite sola: pudo haberse guardado · Qué se ve en la UI: Error de tiempo agotado, sin reintento automático

IDEAS CLAVE

- Servicio caído: el evento espera
- Evento repetido: se ignora
- Acción inválida: 409 con mensaje
- Datos incompletos: 400

QUÉ DECIR

"Esta tabla resume cómo se comporta el sistema ante cada falla que probamos, y qué ve la persona en la interfaz."

"Si un evento llega dos veces, se ignora. Si alguien intenta una acción que no corresponde, como enviar dos veces, recibe un 409 con el motivo. Si faltan datos, un 400 sin guardar nada."

"Y un detalle de diseño en la propia interfaz: si una operación que cambia datos tarda o se corta, no la repite sola, porque el cambio pudo haberse guardado aunque la respuesta no llegara. Ninguna falla queda sin una reacción prevista."

QUÉ HACER (NO SE DICE)

- Mostrar: En Enviadas, Reentregar un evento ya consumido; en Solicitudes, enviar dos veces la misma solicitud. Debe verse: "El aviso no se duplicó" y el diálogo 409.

PARA PASAR

"Esto es lo que aprendimos con la UI como laboratorio."

36. La UI como laboratorio: lo que aprendimos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 5 fallas con reacción prevista
- Métrica: 29 pruebas de la UI
- Diseñar para la falla: Cada falla probada tiene una reacción y un mensaje. Valor: Menos incidentes y menos llamadas a soporte.
- Hacer visible lo asíncrono: Pendientes, entregados y consumidos en una pantalla. Valor: Explicar el sistema a negocio sin abrir el código.
- La demo también enseña: Ayuda con diagramas y botones para provocar fallas. Valor: Material de capacitación para quien llegue al equipo.

IDEAS CLAVE

- Diseñar para la falla, no solo para el éxito
- Hacer visible lo asíncrono
- Una herramienta de demo también enseña

QUÉ DECIR

“Primero: hay que diseñar para la falla, no solo para el camino feliz. La pregunta no es si algo va a fallar, sino cómo se comporta cuando falle.”

“Segundo: lo asíncrono hay que hacerlo visible. Sin la pestaña Enviadas, un evento pendiente sería invisible.”

“Y tercero: una herramienta de demostración también sirve para capacitar. Esta interfaz se puede usar para explicarle el sistema a cualquiera.”

PARA PASAR

“Para cerrar, el valor que deja todo esto.”

Valor y cierre

37. El valor agregado

EN PANTALLA · TABLA DE TRES COLUMNAS

- Pilar: Resiliencia · Qué construimos: Outbox, reintentos e idempotencia · Valor para el equipo: 0 avisos perdidos o duplicados ante caídas
- Pilar: Reproducibilidad · Qué construimos: Docker, Compose y Kubernetes · Valor para el equipo: 1 comando y la misma imagen en todo ambiente
- Pilar: Calidad · Qué construimos: 67 pruebas y pipeline por PR · Valor para el equipo: Errores detectados en minutos, antes de integrar
- Pilar: Trazabilidad · Qué construimos: Logs centralizados, traceId e imagen por SHA · Valor para el equipo: Diagnóstico en minutos y versiones rastreables

- Pilar: Contratos y diseño · Qué construimos: OpenAPI generado y puertos y adaptadores · Valor para el equipo: Integraciones claras y tecnología intercambiable

IDEAS CLAVE

- Resiliencia ante fallas parciales
- Entornos reproducibles
- Calidad en cada cambio
- Trazabilidad de punta a punta

QUÉ DECIR

"Si tuviéramos que resumir el valor en una lámina, sería esta. Construimos resiliencia: ante una caída parcial no se pierde ni se duplica un aviso. Reproducibilidad: un comando y la misma imagen en cualquier ambiente."

"Calidad: sesenta y siete pruebas en cada Pull Request. Trazabilidad: logs centralizados con traceId e imágenes rastreables a su commit. Y contratos claros con un diseño que permite cambiar tecnología sin tocar el negocio."

"Nada de esto es exclusivo de ARKA-Lite: son prácticas que cualquier equipo puede adoptar desde mañana."

PARA PASAR

"Si nos llevamos solo tres ideas, que sean estas."

---

38. Lo que nos llevamos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 0 avisos perdidos o duplicados
- Métrica: 67 pruebas en cada cambio
- Diseñar para la falla: Guardar primero, reintentar e ignorar duplicados. Valor: Sistemas que se recuperan solos.
- Automatizar la confianza: Pruebas, pipeline, imágenes y estado declarado. Valor: Cambiar rápido sin miedo a romper.
- Hacer visible el sistema: Logs, trazas, salud, Swagger y la UI laboratorio. Valor: Diagnosticar y explicar en minutos.

IDEAS CLAVE

- Diseñar para la falla
- Automatizar la confianza
- Hacer visible el sistema

QUÉ DECIR

“Tres ideas. Diseñar para la falla: en un sistema distribuido, algo siempre va a fallar, y la diferencia está en cómo reacciona.”

“Automatizar la confianza: pruebas, pipeline e infraestructura declarada nos permiten cambiar rápido sin miedo.”

“Y hacer visible el sistema: lo que no se ve no se puede diagnosticar ni explicar.”

PARA PASAR

“Gracias por su atención.”

---

39. Gracias

EN PANTALLA

- Un laboratorio que cumplió su objetivo: aprender.
- Proyecto final · Upskilling Backend Java

QUÉ DECIR

“ARKA-Lite fue un laboratorio, y cumplió su objetivo: aprender. Empezamos con un endpoint y terminamos con dos microservicios con su base de datos, empaquetados, desplegados, observados y documentados, que saben comportarse cuando algo falla.”

“Muchas gracias por acompañarnos. Con gusto respondemos sus preguntas.”